

**ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES
BENI MELLAL**

Département Génie Logiciel

RAPPORT DE TRAVAUX PRATIQUES

TP : Git & Node.js

Contrôle de version et gestion de projet

Encadré par : Pr. BE.ELBAGHAZAOUI
Année universitaire 2025 - 2026

Table des Matières

1. Introduction
2. Étape 1 : Initialisation du projet Node.js
3. Étape 2 : Gestion des branches
4. Étape 3 : Fusion (merge)
5. Étape 4 : Résolution de conflits
6. Partie 5 : Rebase
7. Partie 6 : Cherry-pick
8. Étape 6 : Collaboration avec GitHub
9. Synthèse des commandes Git
10. Conclusion

1. Introduction

Ce rapport présente le travail réalisé dans le cadre du TP Git & Node.js du module Génie Logiciel, encadré par le Pr. BE.ELBAGHAZAOUI à l'ENSA de Beni Mellal. Ce TP a pour objectif de maîtriser les commandes essentielles de Git, le système de contrôle de version distribué le plus utilisé dans l'industrie du développement logiciel.

Le TP est structuré autour d'un projet Node.js simple, à travers lequel nous explorons progressivement les concepts clés de Git : l'initialisation d'un dépôt, la gestion des branches, la fusion (merge), la résolution de conflits, le rebase, le cherry-pick, et enfin la collaboration via GitHub.

Git est un outil indispensable pour tout développeur, car il permet de suivre l'historique des modifications, de travailler en équipe de manière organisée, et de gérer efficacement les différentes versions d'un projet.

2. Étape 1 : Initialisation du projet Node.js

2.1 Objectif

Créer un nouveau projet Node.js et initialiser un dépôt Git local avec les fichiers de base.

2.2 Commandes exécutées

```
$ mkdir mon-projet-node  
$ cd mon-projet-node  
$ git init  
$ npm init -y
```

Ces commandes créent un nouveau dossier, initialisent un dépôt Git vide et génèrent un fichier package.json avec les paramètres par défaut.

2.3 Fichiers créés

README.md

```
# Mon Projet Node
```

.gitignore

```
node_modules/
```

index.js

```
console.log('Hello Node');
```

2.4 Premier commit

```
$ git add README.md .gitignore index.js package.json  
$ git commit -m "Initialisation du projet Node"
```

Le fichier .gitignore est essentiel pour exclure le dossier node_modules/ du suivi Git, car il contient les dépendances installées automatiquement par npm et ne doit pas être versionné.

3. Étape 2 : Gestion des branches

3.1 Objectif

Créer une branche de développement (dev) et y ajouter une nouvelle fonctionnalité : la fonction addition.

3.2 Commandes exécutées

```
$ git branch dev  
$ git checkout dev
```

La commande `git branch dev` crée une nouvelle branche nommée `dev`. La commande `git checkout dev` bascule le répertoire de travail sur cette branche. Toutes les modifications suivantes seront enregistrées sur `dev` sans affecter `main`.

3.3 Code ajouté sur la branche dev

```
// index.js (branche dev)  
function addition(a, b) {  
    return a + b;  
}  
  
if (require.main === module) {  
    console.log("Résultat:", addition(5, 3));  
}  
  
module.exports = { addition };
```

3.4 Commit

```
$ git add index.js  
$ git commit -m "Ajout de la fonction addition"
```

Le `module.exports` permet d'exporter la fonction `addition` pour pouvoir la réutiliser dans d'autres fichiers Node.js (tests, etc.).

4. Étape 3 : Fusion (merge)

4.1 Objectif

Fusionner les modifications de la branche `dev` dans la branche `main`.

4.2 Commandes exécutées

```
$ git checkout main  
$ git merge dev
```

La fusion s'effectue ici en mode `fast-forward` car `main` n'a reçu aucun nouveau commit depuis la création de `dev`. Git déplace simplement le pointeur de `main` vers le dernier commit de `dev`, ce qui produit un historique linéaire sans commit de merge.

4.3 Résultat

Après la fusion, la branche main contient désormais la fonction addition. L'historique est propre et linéaire.

Avant merge	Type de merge	Après merge
main : commit initial	Fast-forward	main = dev (même commit)

5. Étape 4 : Résolution de conflits

5.1 Objectif

Provoquer un conflit de fusion entre deux branches ayant modifié le même fichier, puis le résoudre manuellement.

5.2 Création du conflit

Sur la branche feature : ajout de soustraction

```
$ git checkout -b feature
// index.js (branche feature)
function addition(a, b) {
    return a + b;
}
function soustraction(a, b) {
    return a - b;
}
if (require.main === module) {
    console.log("Addition:", addition(5, 3));
    console.log("Soustraction:", soustraction(5, 3));
}
module.exports = { addition, soustraction };
$ git add index.js
$ git commit -m "Ajout de la fonction soustraction"
```

Sur la branche main : ajout de commentaires

```
$ git checkout main
// index.js (branche main)
// addition(a, b) -> retourne la somme de a et b
function addition(a, b) {
    return a + b; // simple addition
}
if (require.main === module) {
    console.log("Résultat:", addition(5, 3));
}
module.exports = { addition };
$ git add index.js
$ git commit -m "Doc: commentaires de addition"
```

5.3 Tentative de fusion

```
$ git merge feature
Auto-merging index.js
CONFLICT (content): Merge conflict in index.js
Automatic merge failed; fix conflicts and commit.
```

5.4 Marqueurs de conflit

Git insère des marqueurs dans le fichier pour délimiter les versions concurrentes :

```
<<<<<< HEAD
// addition(a, b) -> retourne la somme de a et b
function addition(a, b) {
    return a + b; // simple addition
```

```
}  
=====  
function addition(a, b) {  
    return a + b;  
}  
function soustraction(a, b) {  
    return a - b;  
}  
>>>>>> feature
```

La section entre <<<<<< HEAD et ===== correspond à la version de main (avec les commentaires). La section entre ===== et >>>>>> feature correspond à la version de feature (avec soustraction).

5.5 Résolution du conflit

La résolution consiste à combiner les deux versions en gardant les commentaires de main et la fonction soustraction de feature :

```
// addition(a, b) -> retourne la somme de a et b
function addition(a, b) {
    return a + b; // simple addition
}

function soustraction(a, b) {
    return a - b;
}

if (require.main === module) {
    console.log("Addition:", addition(5, 3));
    console.log("Soustraction:", soustraction(5, 3));
}
module.exports = { addition, soustraction };
```

5.6 Finalisation

```
$ git add index.js
$ git commit
# Git propose un message de merge par défaut
```

6. Partie 5 : Rebase

6.1 Objectif

Utiliser git rebase pour réorganiser l'historique des commits et obtenir une histoire linéaire, contrairement au merge qui crée un commit de fusion.

6.2 Création de la branche bugfix

```
$ git checkout -b bugfix
// index.js (bugfix) - addition robuste
function addition(a, b) {
    const x = Number(a), y = Number(b);
    if (Number.isNaN(x) || Number.isNaN(y)) return 0;
    return x + y;
}
$ git add index.js
$ git commit -m "Bugfix: addition robuste (NaN)"
```

6.3 Rebase sur main

```
$ git checkout main
$ git rebase bugfix
```

Le rebase « rejoue » les commits de main au-dessus du dernier commit de bugfix. Contrairement au merge, le rebase produit un historique linéaire sans commit de fusion, ce qui rend l'historique plus lisible.

6.4 Merge vs Rebase

Critère	Merge	Rebase
Historique	Non-linéaire (commit de merge)	Linéaire (pas de commit de merge)
Traçabilité	Conserve l'historique complet	Réécrit l'historique
Usage	Branches partagées	Branches locales / personnelles

7. Partie 6 : Cherry-pick

7.1 Objectif

Appliquer un commit spécifique d'une branche sur une autre branche, sans fusionner l'intégralité de la branche source. C'est utile pour récupérer un bugfix ou une fonctionnalité précise.

7.2 Procédure

```
$ git log --oneline
# Exemple de sortie :
# 3f5c2d3 Bugfix: addition robuste (NaN)
# 1a2b3c4 Ajout de la fonction soustraction
# 9d8e7f6 Doc: commentaires de addition
```

```
$ git checkout feature
$ git cherry-pick 3f5c2d3
```

La commande `git cherry-pick` prend le commit identifié par son hash (3f5c2d3) et l'applique comme un nouveau commit sur la branche courante (feature). Si un conflit survient, il faut le résoudre manuellement puis continuer avec `git cherry-pick --continue`.

7.3 Cas d'usage du cherry-pick

Le cherry-pick est particulièrement utile lorsqu'un correctif critique développé sur une branche doit être appliqué rapidement sur une autre branche (comme main ou une branche de production) sans attendre la fusion complète. Il permet une grande précision dans la gestion des commits.

8. Étape 6 : Collaboration avec GitHub

8.1 Objectif

Connecter le dépôt local à un dépôt distant sur GitHub pour permettre la collaboration en équipe.

8.2 Commandes

```
$ git remote add origin git@github.com:User/mon-projet-node.git
$ git branch -M main
$ git push -u origin main
```

8.3 Configuration de l'identité

```
$ git config --global user.name "VotreNom"
$ git config --global user.email "VotreEmail@example.com"
```

8.4 Workflow de collaboration

Pour travailler en équipe avec Git et GitHub, on adopte généralement le workflow suivant : chaque développeur crée une branche par fonctionnalité (feature/*), pousse ses modifications sur GitHub, puis soumet une Pull Request pour demander la revue de code par ses collègues avant la fusion dans main.

```
# Récupérer les derniers changements
$ git pull origin main

# Pousser ses modifications
$ git push origin main
```

9. Synthèse des commandes Git

Commande	Description
git init	Initialiser un nouveau dépôt Git
git add <fichier>	Ajouter un fichier à la zone de staging
git commit -m "msg"	Enregistrer les modifications avec un message
git branch <nom>	Créer une nouvelle branche
git checkout <branche>	Basculer sur une branche
git merge <branche>	Fusionner une branche dans la branche courante
git rebase <branche>	Rejouer les commits au-dessus d'une branche
git cherry-pick <hash>	Appliquer un commit spécifique sur la branche courante
git log --oneline	Afficher l'historique compact des commits
git remote add origin	Associer un dépôt distant
git push / git pull	Envoyer / récupérer les modifications distantes

10. Conclusion

Ce TP nous a permis de parcourir l'ensemble du workflow Git, depuis l'initialisation d'un dépôt jusqu'à la collaboration via GitHub. Nous avons pu constater l'importance de la gestion des branches pour organiser le développement, la nécessité de savoir résoudre des conflits de fusion, et l'utilité du rebase et du cherry-pick pour maintenir un historique propre et précis.

La maîtrise de Git est une compétence indispensable pour tout ingénieur logiciel. Elle permet de travailler efficacement en équipe, de tracer chaque modification, et de revenir facilement à un état antérieur du code en cas de problème. Le couplage avec Node.js dans ce TP a également permis de découvrir les bases de l'écosystème JavaScript côté serveur.